

Report – Hand-in 1 (Exercise 4.6: Peer-to-Peer Flooding and a Simple Ledger)

1) System design We extended the Peer with:

- Peer set: Each peer keeps a set of known peers (conns). When a peer starts, it connects to an existing peer and gets the list of known peers, then connects to all of them. On receiving a Join message, the new peer is added to the peer set and we open a TCP connection (fully connected network).
- Flooding with deduplication: Each message has a MsgID. Each peer stores in msgHistory which messages it has already flooded and from which peers it received them. When flooding, the message is sent to all peers except those it was received from. On receipt, the message is flooded only if it has not been flooded before. When a new peer R connects to S, S sends all previously seen messages to R (catch-up), so R gets the same history.
- Ledger: Each peer has a local copy (account.Ledger with Accounts map[string]int). Transactions are flooded as message type “Transaction”. When a peer receives a transaction, it is applied locally (ledger.Transaction) with dedup via MsgID. The sender applies the transaction locally immediately, since it does not receive its own flood.

2) How to run the system Open a terminal and run: `cd dissy_da1_1_hand-ins-handin0/peer go run .`

This starts 5 peers, connects them, each sends 5 transactions (same 5 accounts A–E), waits for delivery and checks that all ledgers are identical. Output shows Join/Transaction log.

3) How to run the tests `cd dissy_da1_1_hand-ins-handin0/peer go test`

The tests include: NewPeer, Start, Connect and send, ConnectAndJoinNetwork (same peer set), FloodMessage, Transaction, TestLedgerConvergence (all ledgers equal after flood), TestTransactionDeliveryCount (receiver gets all transactions).

4) Discussion questions (report)

- We implemented deduplicating flooding. The system achieves eventual consistency: each transaction has a unique MsgID and is flooded to all; catch-up ensures new peers get all previous messages. All peers apply the same set of transactions, so ledgers converge (same final balance per account).
- If transactions that would make an account negative are rejected on delivery: as long as all peers apply the same rule and the same set of transactions (or the same order), they still converge to the same ledger. Without total order, the order can vary; if rejection depends on order (e.g. A→B 10 rejected after B→A 5), different peers might end up with different ledgers. With our current “apply all” rule it is not an issue.